Feed-Forward Neural Networks versus Linear Models: Regression of the Runge Function and Classification of Handwritten Digits

Klara Elvira Boström, Vanii Hsin-Ni Yao, Catherine Hughes
(Dated: November 9, 2026)

We investigate if feed-forward neural networks (FFNNs) are worth their additional computational cost compared to simpler methods, by cross-validating and training FFNNs to fit the one-dimensional Runge function and to classify the MNIST handwritten digits. The neural network successfully fits the Runge function without suffering from the boundary oscillations characteristic of the Runge phenomenon, which is a common problem with polynomial regression models, though small datasets can still lead to localized performance limitations. For the classification, a network with one hidden layer of 100 nodes reduces the test error from 7.5% to 2.4% compared to logistic regression, removing about two thirds of its errors and confirming that non-linearity is needed to classify the digits. This comes at the cost of ten times as many parameters, but only 1.8 times the training time.

I. INTRODUCTION

Machine learning has rapidly evolved from a supportive tool in research into a major driver of scientific discovery. Performing tasks such as regression and classification on large datasets is central in a wide range of scientific disciplines. Deep learning is increasingly used for solving complex, high-dimensional problems, particularly for unstructured data such as images, where it can learn useful patterns directly from the raw input [1].

Deep learning relies on multi-layered artificial neural networks designed to automatically learn complex patterns from raw data. Feed-forward neural networks (FFNNs) are the simplest class of artificial neural networks, and they are typically trained with backpropagation and gradient-based optimization. How well a neural network performs depends on several factors, including its structure and how it is trained. Although the networks in this work are shallow, they share the building blocks of deep networks, and studying them gives insight into the basic mechanisms behind deep learning.

We explore the properties of an FFNN by applying it to a regression problem and a classification problem. The Runge function is used to study regression, while the MNIST dataset of handwritten digits [2] is used to study classification. We aim to determine when a neural network is worth its additional computational cost, by studying the advantages and disadvantages of FFNNs in comparison to simpler and cheaper linear methods. For fitting the Runge function, we compare with the traditional linear regression methods OLS, Ridge and Lasso. Polynomial fits of the Runge function are known to oscillate towards the edges of the interval, known as the Runge phenomenon, making it an illustrative example for determining whether a neural network improves the fit. When classifying the MNIST digits, we compare with logistic regression. Since logistic regression is equivalent to our network without a hidden layer, this comparison isolates the effect of the hidden layer and its non-linearity.

Our networks are implemented from scratch, with the backpropagation verified against automatic differentiation in `JAX`. For the Runge function, we explore the effect of varying the number of hidden layers, the activation functions, the form of regularization and the gradient descent methods, as well as select optimal learning rate and regularization strength via cross-validation before performing the final fit with this configuration. For the MNIST digits, we similarly choose the learning rate and the regularization strength by cross-validation for both the network and logistic regression, before selecting the width of the network with a validation curve. The final models are evaluated once on a held-out test set, and for the network we study the confusion matrix and per-class metrics to see which digits are difficult to classify. Finally, we weigh the gain in accuracy against the additional cost of the network, in terms of the number of parameters, the computational cost and the training time.

In Sec. II, we present an overview of FFNN's and their composition, a note on logistic regression and classification metrics and the basics of cross-validation. In Sec. III we describe the details of our implementation. Sec. IV presents and discusses our results. Finally, Sec. V provides a summary and a final outlook.

All code developed for this report is available via our GitHub repository. [1]

II. THEORY

We first introduce the theoretical concepts used in the analysis, followed by the specific choices and methods used in our implementation.

Feed-Forward Neural Networks

A Feed-forward neural network (FFNN) consists of an input layer, one or more hidden layers, and an output layer. Each layer contains artificial neurons, or nodes,

[1] https://github.com/elvirabostrom/FYS-STK4155

connected to the neurons in the previous layer. Information moves from the input layer through the hidden layers to the output layer, without forming cycles [3].

We call the output of each layer $\mathbf{a}^{(l)} = [a_1^{(l)} \; a_2^{(l)} \; \cdots \; a_{M_l}^{(l)}]^T$, with $l = 0, 1, \ldots, L$ denoting the number of the layer. The output of layer l is then the input of layer $l+1$, and we define $\mathbf{a}^{(0)} = \mathbf{x}$ to be a raw sample going into the input layer. Each element $a_i^{(l-1)}$, with $i = 1, 2, \ldots, M_{l-1}$ labeling the nodes, enters each node of the next layer, where they are multiplied by a weight $w_{ij}^{(l)}$, a bias $b_j^{(l)}$ is added and they are summed, giving the net input [4]

$$z_j^{(l)} = \sum_{i=1}^{M_{l-1}} w_{ij}^{(l)} a_i^{(l-1)} + b_j^{(l)}, \tag{1}$$

where j denotes a node in the current layer. In matrix-vector form this is given by

$$\mathbf{z}^{(l)} = \left(\mathbf{W}^{(l)}\right)^T \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \tag{2}$$

where $\mathbf{W}^{(l)}$ is the weight matrix, $\mathbf{a}^{(l-1)}$ is the output from the previous layer, and $\mathbf{b}^{(l)}$ is the bias vector. The output is calculated from the net input using a non-linear activation function f,

$$\mathbf{a}^{(l)} = f\left(\mathbf{z}^{(l)}\right). \tag{3}$$

With n samples and m features, the raw data is given as batched samples in the shape of a $n \times m$ matrix $\mathbf{X} = \mathbf{A}^{(0)}$. The net input then takes the form

$$\mathbf{Z}^{(l)} = \mathbf{A}^{(l-1)}\mathbf{W}^{(l)} + \mathbf{b}^{(l)}, \quad \mathbf{A}^{(l)} = f(\mathbf{Z}^{(l)}), \tag{4}$$

where the weight matrix is now non-transposed and has the shape (input, output). The bias is added by broadcasting. For simplicity, the theory of this report is formulated in the per-sample form.

Two main sweeps are performed in a feed forward neural network. First the feed forward pass, with randomized scaled weights, letting data flow from the input layer to the output layer computing the net inputs and producing the final output $\mathbf{a}^{(L)} = \hat{\mathbf{y}}$. A cost function then compares the computed output to the true target value $\mathbf{y}$. The measured error is then propagated backwards by using the chain rule and gradient descent to update the weights and biases, this is backpropagation. The feed forward pass, error measure and back propagation procedure is repeated, and the update of the weights and biases is the network learning. All of these components are summarized in Alg. 1, and will be discussed in further detail in the following.

Algorithm 1 Feed-forward neural network

Define architecture and activation functions ▷ L, M_l
Define cost function, hyperparameters and gradient descent method
Initialize weights and biases ▷ $\mathbf{W}^{(l)}$, $\mathbf{b}^{(l)}$ for all l
Scale input and target ▷ $\mathbf{x}$, $\mathbf{y}$

while iterations $<$ $\max_{\text{it}}$ **and/or** stopping criterion not reached **do**
  Feed-forward
    Feed-forward pass ▷ $\mathbf{a}^{(L)} = \hat{\mathbf{y}}$
    Compute errors ▷ Compare $\hat{\mathbf{y}}$ againts $\mathbf{y}$
  Backpropagation
    Backpropagate errors
    Update $\mathbf{W}^{(l)}$, $\mathbf{b}^{(l)}$ by gradient descent
end while
return Trained model $\mathbf{W}^{(l)}$, $\mathbf{b}^{(l)}$ for all l

Activation Functions

Activation functions introduce non-linearity, allowing neural networks to learn complex relationship between inputs and outputs. Using only linear activation functions would be the same as having no hidden layers, because they collapse to nothing but linear regression or logistic regression [3]. The network can then only learn linear relationships in the data, and curved non-linear trends are overlooked. The derivatives of the activation functions are used for the backpropagation, and it is thus important that they are differentiable almost everywhere [3]. Multiple activation functions are available, here we list some of them and their most important properties.

The Sigmoid activation function is applied element-wise as

$$\sigma(z_j) = \frac{1}{1 + e^{-z_j}}, \tag{5}$$

and maps input values to the interval between 0 and 1. Its derivative, listen in Tab. I, approaches zero for inputs of large positive and negative magnitude. This can lead to the vanishing gradient problem and slow down learning [5].

The Rectified Linear Unit (ReLU) does not suffer from vanishing gradients for positive inputs. It is given by

$$\text{ReLU}(z_j) = \max(0, z_j), \tag{6}$$

where the output is zero for negative inputs and positive inputs are passed through unchanged. It can suffer from the dying ReLU problem, where a negative input gives a zero-gradient (see Tab. I) and neurons become inactive and stop learning.

Leaky ReLU addresses the dying ReLU limitation by introducing a small positive slope for negative inputs:

$$\text{LeakyReLU}(z_j) = \begin{cases} z_j, & z_j > 0, \\ \alpha z_j, & z_j \leq 0, \end{cases} \tag{7}$$

where $\alpha \approx 0.01$ is a fixed parameter. This allows gradients to remain non-zero and reduces the risk of inactive neurons during training.

The best choice depends on the specific network structure, how fast it should be trained and if the vanishing

gradient problem or dying ReLU are present [5]. ReLU is used as a starting point before trying LeakyReLU, as LeakyReLU provides no advantage unless the dying ReLU is an issue [5]. For fitting smooth functions, sigmoid or the tanh function [5] are often preferred over ReLU, as ReLU's second derivative is zero.

In the output layer, the final output $\hat{\mathbf{y}}$ is computed by a specific activation function. For a sample $\mathbf{x}$ and K classes, the softmax function gives the predicted probability that the sample belongs to class k. $M_L = K$ and the nodes are labeled by the class index $k = 0, \ldots, K-1$ instead of j:

$$a_k^{(L)} = p_k = \frac{\exp\left(z_k^{(L)}\right)}{\sum_{k'=0}^{K-1} \exp\left(z_{k'}^{(L)}\right)}, \tag{8}$$

where $z_k^{(L)}$ is then the net input of the output layer for class k. The predicted probabilities are positive and sum to one over the K classes. For a batch of n samples, the softmax function is applied row-wise to the $n \times K$ matrix $\mathbf{Z}^{(L)}$, giving the $n \times K$ matrix $\hat{\mathbf{Y}}$, where element (s, k) is the predicted probability that sample s belongs to class k.

For regression, the output layer has a single node, $M_L = 1$, and the linear activation function returns the net input unchanged,

$$\hat{\mathbf{Y}} = \mathbf{A}^{(L)} = \mathbf{Z}^{(L)}, \tag{9}$$

of dimensions $n \times 1$, containing one prediction per sample.

Cost Functions

Cost functions measure the difference between predicted and true values as a mean over all samples. We collect the weights and biases of all layers in one parameter $\boldsymbol{\theta} = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}$ giving

$$C(\boldsymbol{\theta}) = \frac{1}{n} \sum_{s=1}^{n} \mathcal{L}(\mathbf{y}_s, \hat{\mathbf{y}}_s(\boldsymbol{\theta})) + \lambda \Omega(\boldsymbol{\theta}), \tag{10}$$

where $\mathcal{L}$ is the per-sample cost function and λ determines the strength of the regularization term $\Omega(\boldsymbol{\theta})$. The choice of cost function depends on the problem, with the mean squared error (MSE) commonly used for regression and cross-entropy (CE) for classification.

The MSE, Eq. 10 with

$$\mathcal{L}_{\text{MSE}}(y_s, \hat{y}_s) = (\hat{y}_s - y_s)^2, \tag{11}$$

measures the average squared difference between predicted and true target values over n samples, and $M_L = 1$. It is suitable for predicting continuous values and is paired with the linear activation function in the output layer. The predictions of a regression problem should be able to take any real value, and the output is therefore not restricted to a limited range.

The multiclass cross entropy for K classes is given by

$$\mathcal{L}_{\text{CE}}(\mathbf{y}_s, \hat{\mathbf{y}}_s) = -\sum_{k=0}^{K-1} y_{sk} \log p_k, \tag{12}$$

where log is the natural logarithm, the probability is given by the softmax function (Eq. 8) and $\mathbf{y}_s$ is the one-hot encoding, where the elements are

$$y_{sk} = \begin{cases} 1, & \text{if sample } s \text{ belongs to class } k, \\ 0, & \text{else.} \end{cases} \tag{13}$$

The cost functiin is minimized with respect to the weights and biases during training, giving the optimal parameters

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} C(\boldsymbol{\theta}) \tag{14}$$

$$= \arg\min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{s=1}^{n} \mathcal{L}(\mathbf{y}_s, \hat{\mathbf{y}}_s(\boldsymbol{\theta})) + \lambda \Omega(\boldsymbol{\theta}). \tag{15}$$

Regularization

The regularization term $\Omega(\boldsymbol{\theta})$ reduces overfitting by adding a penalty term to the cost function. Usually, only the weights are regularized. Let $\mathbf{w}$ denote all weights of the network collected in a single vector, excluding the biases. ℓ_1 regularization can reduce some weights to zero, performing feature selection, while ℓ_2 regularization encourages smaller weights [3] [6]. For ℓ_2 regularization, the penalty $\lambda \|\mathbf{w}\|_2^2$ is added to the cost function, contributing a $2\lambda\mathbf{w}$ term to the gradient. Similarly, ℓ_1 regularization adds $\lambda \|\mathbf{w}\|_1$, contributing a $\lambda \operatorname{sgn}(\mathbf{w})$ term for nonzero weights. In both cases, the hyperparameter λ controls the strength of regularization. At zero, a subgradient is used.

In contrast to linear regression methods [6], there is no closed form solutions for the optimal weights and biases, as the network is a composition of non-linear functions. Because of this gradient descent methods are used for the update step during backpropagation.

Backpropagation

The backpropagation part of Alg. 1 is itself an algorithm used to calculate the gradients of the cost function with respect to the weights and biases in a neural network. It applies the chain rule to propagate errors backward from the output layer through the hidden layers [3]. The total gradient is

$$\nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}) = \frac{1}{n} \sum_{s=1}^{n} \nabla_{\boldsymbol{\theta}} \mathcal{L}(\mathbf{y}_s, \hat{\mathbf{y}}_s(\boldsymbol{\theta})) + \lambda \nabla_{\boldsymbol{\theta}} \Omega(\boldsymbol{\theta}), \tag{16}$$

which is quite difficult to evaluate as a whole. The following four equations, derived in App. A, explain the

backpropagation, where $\boldsymbol{\delta}^{(l)}$ contains the errors of the nodes in a layer, f' is the derivative of the activation function and i and j denote neuron indices. The symbol $\odot$ represents element-wise multiplication.

1. The error in the output layer is calculated using the derivative of the cost function and the activation function,

$$\boldsymbol{\delta}^{(L)} = f'(\mathbf{z}^{(L)}) \odot \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}}. \tag{17}$$

2. The error is propagated backward through the hidden layers using the error from the next layer,

$$\boldsymbol{\delta}^{(l)} = (\mathbf{W}^{(l+1)}\boldsymbol{\delta}^{(l+1)}) \odot f'\left(\mathbf{z}^{(l)}\right). \tag{18}$$

3. The gradient with respect to each bias is equal to the error of the corresponding neuron,

$$\frac{\partial \mathcal{L}}{\partial {b_j}^{(l)}} = \delta_j^{(l)}. \tag{19}$$

4. The gradient with respect to each weight is calculated by multiplying the error of the neuron by the activation from the previous layer,

$$\frac{\partial \mathcal{L}}{\partial {w_{ij}}^{(l)}} = \delta_j^{(l)} {a_i}^{(l-1)}. \tag{20}$$

Eq. 17 assumes an element-wise activation function, which does not hold for softmax, since each probability p_k depends on all net inputs in the output layer. When the derivatives of the cross-entropy and the softmax function are combined, the factors of p_k cancel, and the error reduces to

$$\boldsymbol{\delta}^{(L)} = \hat{\mathbf{y}} - \mathbf{y}, \tag{21}$$

which replaces Eq. 17. Here, $\mathbf{y}$ is the one-hot encoded targets (elements as in Eq. 13) and $\hat{\mathbf{y}}$ is the vector of softmax probabilities p_k.

With regularization, the derivative of the penalty is added to the weight gradient in Eq. 20,

$$\frac{\partial C}{\partial w_{ij}^{(l)}} = \frac{1}{n}\sum_{s=1}^{n} \delta_{sj}^{(l)} a_{si}^{(l-1)} + \begin{cases} 2\lambda w_{ij}^{(l)}, & \ell_2, \\ \lambda \operatorname{sgn}\left(w_{ij}^{(l)}\right), & \ell_1, \end{cases} \tag{22}$$

while the bias gradient in Eq.19 is unchanged. The derivatives used in our implementation are given in Tab. I, and the derivation for the cost functions can be found in App. B. The derivatives of the activation functions are taken from [3].

The gradient computed by backpropagation is used to update the weights and biases. Plain gradient descent [6] and a learning rate γ gives the update

$$w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \gamma \delta_j^{(l)} a_i^{(l-1)}, \tag{23}$$

$$b_j^{(l)} \leftarrow b_j^{(l)} - \gamma \delta_j^{(l)}. \tag{24}$$

TABLE I: **Derivatives inserted into the backpropagation equations for a single sample.** Here, p_k is the softmax probability for class k, and $I_{kk'}$ is element (k, k') of the identity matrix.

Function	**Derivative**
MSE	$\frac{\partial \mathcal{L}}{\partial \hat{y}} = 2(\hat{y} - y)$
Cross-entropy	$\frac{\partial \mathcal{L}}{\partial p_k} = -\frac{y_k}{p_k}$
Sigmoid	$\sigma'(z_j) = \sigma(z_j)\left[1 - \sigma(z_j)\right]$
ReLU	$f'(z_j) = 1$ for $z_j > 0$; 0 for $z_j \leq 0$
Leaky ReLU	$f'(z_j) = 1$ for $z_j > 0$; α for $z_j \leq 0$
Linear	$f'(z_j) = 1$
Softmax	$\frac{\partial p_k}{\partial z_{k'}} = p_k\left(I_{kk'} - p_{k'}\right)$

The learning rate can be constant or updated in an adaptive scheme such as Adam or RMSProp [3] [6]. This can be combined with stochastic gradient descent (SGD) [3][6], which addresses the prohibitive computational cost of standard gradient descent on large datasets by approximating the gradient using mini-batches of size M. At the start of an epoch, the training data is shuffled and split into mini-batches, so one epoch is one pass through the data, i.e. n/M updates.

Logistic Regression

Logistic regression is used for classification and corresponds to a neural network with no hidden layers, so that $L = 1$ and $M_1 = K$. Everything else is the same. Backpropagation reduces to a single layer and the model is limited to describing linear relationships. Unlike for a network with hidden layers, the cross-entropy is convex in the weights and biases, so every local minimum is also a global minimum [3].

Logistic regression can be used as a baseline for evaluating a neural network. With m features, the model has only $K(m + 1)$ parameters, and a feed-forward pass costs $\mathcal{O}(mK)$ operations per sample, compared to $\mathcal{O}(\sum_l M_{l-1}M_l)$ for the general network [3]. If the neural network does not clearly perform better than this baseline, the added complexity of a hidden layer is not worth the cost [3].

Classification Metrics

The accuracy is the number of correctly predicted targets y_i in a classification problem, divided by the total number of targets,

$$\text{Accuracy} = \frac{\sum_{i=1}^{n} I(y_i = \hat{y}_i)}{n}, \tag{25}$$

where I is 1 if $y_i = \hat{y}_i$ and 0 otherwise. Accuracy alone is not a good estimate of model performance, because if the true targets are imbalanced across the classes, the accuracy can be high even if the model only predicts some of the classes correctly [3]. The measure becomes more meaningful in comparison to, e.g., the logistic regression baseline.

The confusion matrix [3] is a table of the model's predictions against the true targets for each class. It shows which classes are confused with each other and thus gives a detailed picture of the mistakes the model makes. From it, we can compute the precision, recall and F1 score for each class k,

$$\text{Precision}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k}, \tag{26}$$

$$\text{Recall}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k}, \tag{27}$$

$$\text{F1}_k = \frac{2\,\text{Precision}_k\,\text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}, \tag{28}$$

where TP_k is the number of samples in class k that are correctly predicted, FP_k is the number of samples from other classes that are predicted as class k, and FN_k is the number of samples in class k that are predicted as another class. The precision measures the fraction of predictions of class k that are correct. The recall measures the fraction of samples in class k that the model identifies. The F1 score is the harmonic mean of the two, and is high only if both are high.

Cross-Validation and Model Selection

To select the hyperparameters of a neural network, we minimize the cross-validation (CV) error [6]. The training set is shuffled and split into K folds, not to be confused with the K classes from earlier. Each fold is held out one time as a temporary test set, while the model is fitted on the remaining folds and then used to predict the held-out fold. The CV error is the mean error over all held-out folds,

$$\text{CV} = \frac{1}{K}\sum_{k=1}^{K} E_k, \tag{29}$$

where E_k is the validation error on fold k. For regression it is the MSE, while for classification it is $1 - \text{Accuracy}$. It measures how well the training procedure performs on training sets of size $n(K-1)/K$.

It is reported with its standard error (SE),

$$\text{CV} \pm \text{SE}_K, \tag{30}$$

where $\text{SE}_K = s/\sqrt{K}$ and s is the standard deviation of the K fold errors. The SE is optimistic because it assumes independency between the folds, but the training sets in different folds share a fraction of their points and the different E_k are not independent.

The minimum of the CV curve is usually not clear-cut, as other hyperparameter values can give a CV error within one SE of it. In that case, we choose the simplest model, e.g. the largest penalty λ, and this is called the one-SE rule. The final model is obtained by training with the selected hyperparameters on the full training set, without any splitting. Finally the test set is used to evaluate the choosen model.

III. METHODS

Initialization of the Neural Network

This report only deals with fully connected FFNNs, where all nodes of one layer are connected to all nodes of the next. Our input is in batched form, with the forward pass following Eq. 4, while all other per-sample equations and derivations still apply.

Our FFNN models are initialized by choosing the number of nodes and the activation function for each layer, the cost function, and the regularization (ℓ_1, ℓ_2 or `None`) with strength λ. Sigmoid, ReLU and Leaky ReLU are considered as activation functions for the hidden layers. When fitting the Runge function, the output activation function is the linear function and the cost function is the MSE. For the multiclass MNIST problem, the output activation function is the softmax, returning a probability for each digit 0 through 9, and the cost function is the cross entropy.

The biases $\mathbf{b}^{(l)}$ are initialized with all M_l elements set to 0.01. This small positive value is chosen to reduce the risk of dying ReLU and, for simplicity, is used for all layers. The weights are initialized randomly so that the nodes in a layer do not all compute the same output, and are scaled according to the activation function of the layer [3]. Xavier/Glorot initialization sets the variance of each weight w_{ij} to

$$\text{Var}(w_{ij}) = \frac{2}{M_{l-1} + M_l}, \tag{31}$$

and is used for all layers except those with ReLU or Leaky ReLU, where we use He initialization,

$$\text{Var}(w_{ij}) = \frac{2}{M_{l-1}}, \tag{32}$$

which for equal layer sizes is twice the Xavier variance, compensating for ReLU setting approximately half of its inputs to zero [3]. The weights in $\mathbf{W}^{(l)}$ ($M_{l-1} \times M_l$) are drawn from a normal distribution with mean zero and standard deviation $\sqrt{\text{Var}(w_{ij})}$, using seed 2026.

To verify our backpropagation, we compare the computed gradients with those from automatic differentiation in `JAX` (`jax.grad`), which applies the chain rule to the operations in the code and is accurate to machine precision [7].

Data Sets

The Runge function

$$f(x) = \frac{1}{1 + 25x^2} \tag{33}$$

is sampled at $n = 100$ points x_i drawn uniformly from the interval $[-1, 1]$. The targets are $y_i = f(x_i) + \epsilon_i$, where the noise ϵ_i is drawn from a normal distribution with mean zero and standard deviation $\sigma = 0.1$. The data are generated and shuffled using a random number generator with and then split into 80 training points and 20 test points. For the preliminary investigations, the seed is set to 2026. For cross-validation and final evaluation, a new dataset is generated using seed 123. The test set of this dataset is reserved exclusively for the final evaluation.

The MNIST set of handwritten digits [2] contains 70 000 images of 28×28 pixels valued between 0 and 255. It comes with a split into 60 000 training images and 10 000 test images. We scale the pixels to the interval $[0, 1]$ by dividing by 255, and flatten each input image to a row of 784 features. The input matrix $\mathbf{X}$ is then of dimension $60\,000 \times 784$. The training targets $\mathbf{y}$ are stored as a vector of length 60 000. Before training the model, the targets are organized as one-hot matrices. The test set is not touched until the final evaluation.

Preliminary Model Exploration

Prior to cross-validation, we conduct a preliminary analysis to select the most promising gradient descent method and investigate how the networks performance is affected by various settings. We evaluate full-batch (plain) gradient descent alongside mini-batch Stochastic Gradient Descent (SGD) using the Adam and RMSprop optimizers (batch size $M = 16$, 2000 epochs and a convergence tolerance 10^{-7}). We test both single- and two-hidden-layer architectures across learning rates $\gamma \in \{0.0005, 0.001, 0.005, 0.01\}$ without regularization. We also investigate the use of sigmoid, ReLu and LeakyReLU in the hidden layers. The data is scaled before training and scaled back prior to evaluation. Models are evaluated by calculating the MSE on the 20 test data points (seed 2026) and by visually inspecting the fitted curves. To evaluate visual fit, predictions are generated on a fine grid of 500 uniformly spaced points over $[-1, 1]$ and plotted alongside the test data. Based on these results, we identify a promising optimizer, learning rate, and network depth for the subsequent experiments. Some of these choices are further investigated and refined through cross-validation.

Cross-Validation and Hyperparameter Selection

For the regression problem, to obtain an independent evaluation, we generate the new dataset with seed 123. We perform a grid search using scikit-learn's `GridSearchCV` with 5-fold cross-validation (`KFold`), with shuffling and seed 123, over $\lambda \in \{10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 0.01\}$, one and two hidden layers with 50 nodes and learning rates $\gamma \in \{0.0005, 0.001, 0.003, 0.005, 0.01\}$. Our network is wrapped in a scikit-learn compatible estimator class, inheriting from `BaseEstimator` and `RegressorMixin`, so that it can be used with scikit-learn's model selection tools. Furthermore, we use a scikit-learn `Pipeline` to encapsulate the preprocessing and model training steps to prevent data leakage.

For the classification, the grid search is performed using scikit-learn's `GridSearchCV` with 5-fold stratified cross-validation (`StratifiedKFold`), over $\lambda \in \{10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 0.01\}$ and $\gamma \in \{10^{-4}, 10^{-3}, 3 \cdot 10^{-3}, 0.01, 0.03\}$, scored by accuracy. The network is wrapped in a custom estimator, inheriting from `BaseEstimator` and `ClassifierMixin`. The grid search is first done for a network with one hidden layer of 50 nodes with the sigmoid activation function. The chosen model is selected by the one standard error rule, CV accuracy $\geq$ best CV accuracy $-$ best SE, then the highest λ, then the highest CV accuracy. With the chosen λ and γ fixed, we compute a validation curve over $5, 10, 25, 50$ and 100 nodes in the hidden layer using scikit-learn's `validation_curve` with the same folds. The chosen width also follows the one SE rule. Logistic regression is implemented as the same network, but with no hidden layers, using the same folds and parameter grid, with λ and γ selected in the same manner. All models are trained with Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$) and ℓ_2 regularization, for exactly 10 epochs with minibatches of size 130, reshuffled each epoch with seed 2026. The number of epochs is kept low to reduce the computation time, and no criterion for convergence or early stopping is used.

After Choosing the Models

Using the optimal hyperparameters selected via cross-validation, alongside the preferred optimizer and activation function identified in our preliminary analysis, we retrain the final model on the full training set (seed 123). The model is then evaluated once on the separate test set by calculating the MSE and visually inspecting the fitted curves against the closed-form OLS results from our previous work. In our previous work, OLS with a polynomial degree $d = 10$ was found to be the preferred model over Ridge and Lasso for fitting the Runge function [6].

The final classification models are trained on all 60 000 training images with the chosen λ, γ and width, with the same training procedure as done in cross-validation. The final models are then evaluated once on the test set. We determine a baseline accuracy with the majority class, and compare logistic and network. For the network with a hidden layer, we compute the absolute and row-normalized confusion matrix with scikit-learn's

`ConfusionMatrixDisplay` and report the precision, recall and F1 with `classification_report`.

Tools

We use `scikit-learn` [8] for model selection and metrics, `JAX` [7] to test our gradients, `NumPy` [9] for numerics, and `Matplotlib` [10], `Seaborn` [11] and `Pandas` [12] for visualizations and storing results. AI tools have been utilized in this study, and a formal declaration can be found in App. C.

IV. RESULTS AND DISCUSSION

Choosing a Gradient Descent Method for Regression

Tables II and III compare MSE values across gradient descent methods and learning rates for the single-hidden-layer and two-hidden-layer architectures, respectively. Figures 1 and 2 show fitted curves for the different learning rates for networks trained with Adam and RMSprop, respectively.

For the single-hidden-layer setup trained with Adam, the learning rate $\gamma = 0.005$ yields the lowest preliminary test MSE (0.0231) and provides the most visually accurate fit to the Runge function among the tested rates. Lowering the learning rate to $\gamma = 0.001$ yields a higher MSE (0.0246) and introduces noticeable underfitting. At an even lower learning rate ($\gamma = 0.0005$), the optimizer fails to fit the target function entirely, predicting a flat horizontal line with a high MSE. Conversely, the larger $\gamma = 0.01$ provides a decent visual fit, but results in a higher MSE (0.0266) relative to the optimal model. A similar trend is observed for the single-hidden-layer model trained with RMSprop, where $\gamma = 0.005$ yields the lowest test MSE (0.0223). Although this test score is lower than the best score achieved by Adam for the single-layer setup, visual inspection shows that Adam produces a superior fit to the Runge function at this learning rate. It should also be noted that RMSProp fails to meet the convergence criterion within the 2000-epoch limit across all tested learning rates, with the exception of $\gamma = 0.0005$, which fails to fit the target function altogether.

For the two-hidden-layer setup trained with Adam, $\gamma = 0.0005$ produces the lowest MSE (0.0186). However, the corresponding plot shows an insufficient fit. The learning rate $\gamma = 0.001$ achieves the second-lowest MSE (0.0220) while providing the visually superior fit. Higher rates of $\gamma = 0.005$ and $\gamma = 0.01$ fail to converge within 2000 epochs and result in visible overfitting. For the two-hidden-layer network trained with RMSprop, both $\gamma = 0.0005$ and $\gamma = 0.001$ produce visually good fits, with $\gamma = 0.0005$ achieving the lowest MSE (0.0219). However, we see that both fail to converge within 2000 epochs. Rates of $\gamma = 0.005$ and $\gamma = 0.01$ result in clear overfitting and also fail to converge. Plain gradient descent performs poorly across all single- and two-layer configurations. In all cases it fails to fit the function, yielding high MSE scores and flat horizontal lines as predictions. Thus, we do not include the plots here.

Overall, several key trends emerge from these results. First, the two-hidden-layer architecture outperforms the single-layer setup, achieving both lower MSE scores and superior visual fits, though care must be taken to avoid overfitting. Second, while Adam and RMSprop achieve comparable test MSE scores, RMSprop consistently fails to converge within the 2000-epoch limit across all successful configurations, rendering its performance unreliable. Adam's superior performance over RMSprop is anticipated given the form of the Runge function, which has a sharp central peak and flat tails. Adam pairs adaptive learning rate scaling with momentum, which allows it to maintain velocity through flat regions.

Finally, relying solely on MSE to select the optimal model can be misleading, as certain configurations yield low error despite inadequate visual fit. Both overfitting and poor fitting can produce deceptively low MSE values. In overfitting, this stems from excessive local curvature, while in poor fitting, it can arise when evaluating on a small test set. With a test set of only 20 points, every test point accounts for 5% of the total test MSE. A model may fail to capture the underlying function, yet still yield a low MSE by accidental proximity with a few high-variance points. So far, the best balance of visual fit and low error is achieved by the two-hidden-layer setups using RMSprop at $\gamma = 0.0005$ (MSE = 0.0219) and Adam at $\gamma = 0.001$ (MSE = 0.0220). Because Adam converges within 1170 epochs, unlike RMSprop, which hits the 2000-epoch limit, the two-layer Adam setup is selected for further work.

Preferred Activation Function for Regression

Figure 3 shows fitted curves obtained using ReLU and LeakyReLU for both single-hidden-layer and two-hidden-layer architectures. Although ReLU and LeakyReLU achieve relatively low MSE values ranging between 0.0215 and 0.0233, their resulting approximations exhibit a jagged, piecewise-linear shape. This is consistent with the note in our theory, that both activation functions have a second derivative of zero. In contrast, the sigmoid function is smooth and has a non-zero second derivative, making it better suited for capturing the smooth curvature of the Runge function. Subsequent analyses focus exclusively on the sigmoid activation function.

Selection of Hyperparameters and Final Fit

The cross-validation run identifies the two-hidden-layer architecture as superior, consistent

TABLE II: **One hidden layer: test MSE comparison across optimizers and learning rates.** Results for an unregularized neural network with 1 hidden layer (50 nodes) trained over 2,000 epochs (convergence criterion 10^{-7}) to fit the Runge function. Models are evaluated across $\gamma \in \{0.0005, 0.001, 0.005, 0.01\}$ using SGD with Adam, SGD with RMSprop (both using mini-batch size $M = 16$), and full-batch plain Gradient Descent. Adam and RMSprop achieve comparable MSE scores, but RMSprop fails to converge within the epoch limit. Plain GD performs poorly.

	Adam		**RMSprop**		**Plain GD**	
Learning Rate (γ)	MSE	Epochs	MSE	Epochs	MSE	Epochs
0.0005	0.1189	169	0.1182	59	0.1241	2000*
0.001	0.0246	944	0.0223	2000*	0.1220	2000*
0.005	0.0231	693	0.0246	2000*	0.1202	1121
0.01	0.0266	1430	0.0262	2000*	0.1201	686

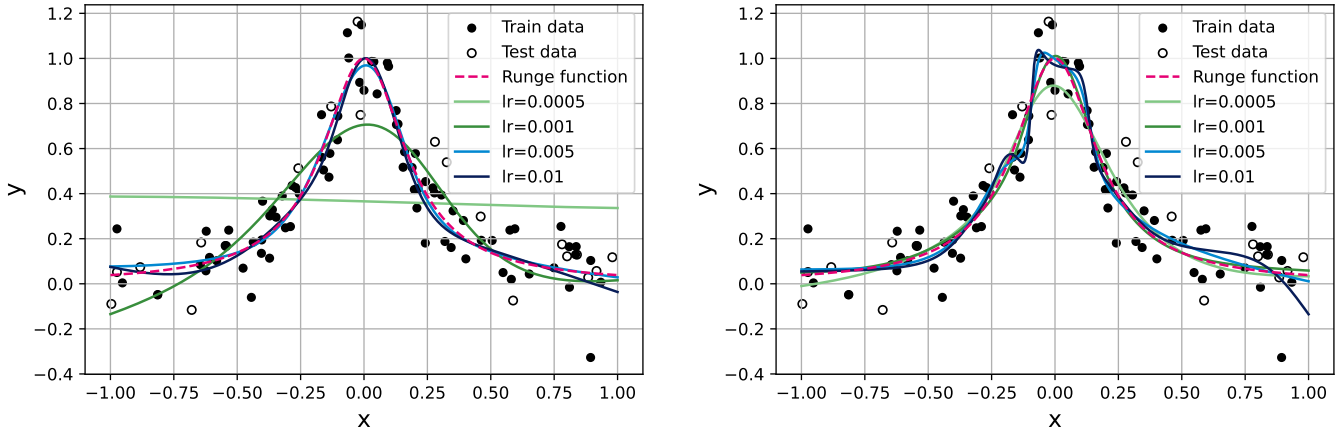
**Hit maximum epoch limit without meeting tolerance threshold.*

TABLE III: **Two hidden layers: test MSE comparison across optimizers and learning rates.** Results for an unregularized neural network with 2 hidden layers (50 nodes each) trained over 2,000 epochs (convergence criterion 10^{-7}) to fit the Runge function. Models are evaluated across $\gamma \in \{0.0005, 0.001, 0.005, 0.01\}$ using SGD with Adam, SGD with RMSprop (both using mini-batch size $M = 16$), and full-batch plain Gradient Descent. Adam and RMSprop achieve comparable MSE scores, but RMSprop fails to converge at any of the learning rates. Plain GD performs poorly.

	Adam		**RMSprop**		**Plain GD**	
Learning Rate (γ)	MSE	Epochs	MSE	Epochs	MSE	Epochs
0.0005	0.0186	1230	0.0219	2000*	0.1208	390
0.001	0.0220	1170	0.0229	2000*	0.1207	206
0.005	0.0224	2000*	0.0246	2000*	0.1206	46
0.01	0.0252	2000*	0.0257	2000*	0.1205	23

**Hit maximum epoch limit without meeting tolerance threshold.*

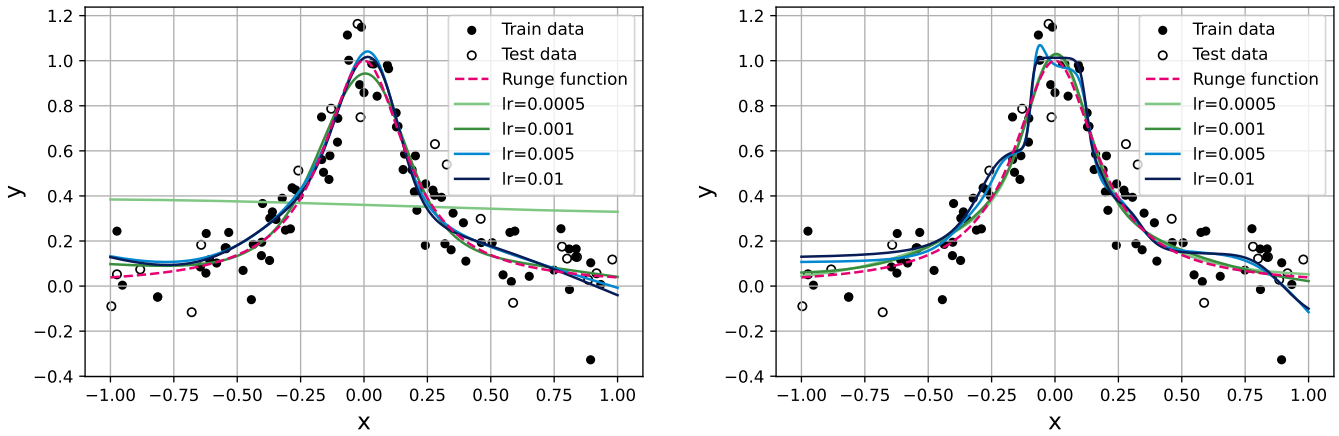
with our observations from the plots in the preliminary analysis. Among the learning rates $\gamma \in \{0.0005, 0.001, 0.003, 0.005, 0.01\}$ and regularization penalties $\lambda \in \{0.0, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 0.01\}$, it identifies $\lambda = 0.0$, $\gamma = 0.005$ as the optimal hyperparameters, which corresponds to an unregularized case. The final fit on the Runge data (Figure 4) is performed using these optimal hyperparameters, yielding a test MSE of 0.0088. The "bulge" on the left, where the fit deviates from the true function, could be the model's attempt to overfit, which is consistent with the preliminary analysis where the model with the same configuration and learning rate (but a different data set) also exhibited overfitting behavior. It should be noted that in that exact region ($x \in [-0.5, -0.22]$), there is a noticeable gap in the training data points, which may explain why the model struggles to fit adequately. Such shortcomings are characteristic of small data sets. With a larger number of samples, a denser distribution of points would minimize local fitting errors.



(a) Single hidden layer.

(b) Two hidden layers.

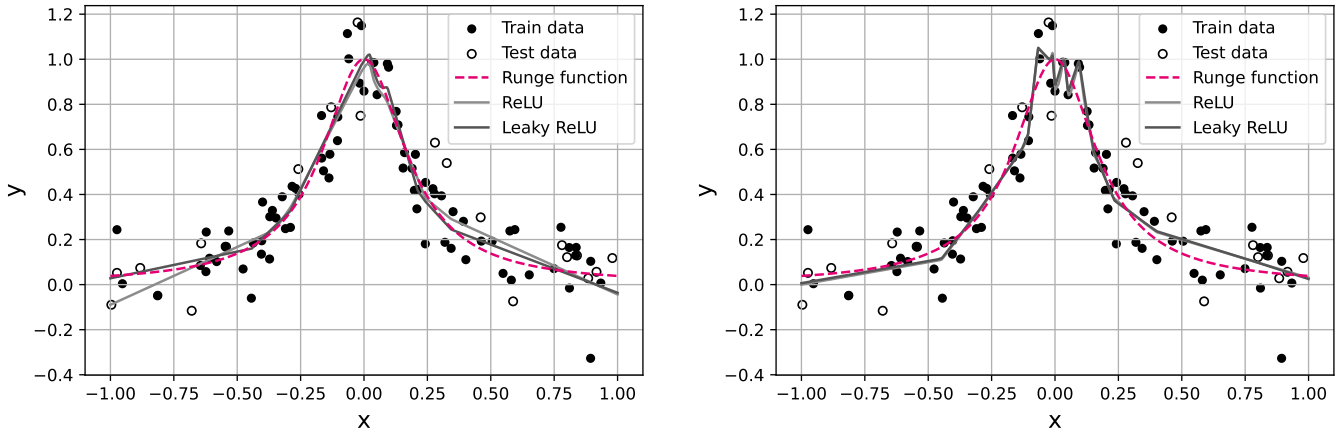
FIG. 1: **Comparison of learning rates for fitting the Runge function using SGD with Adam optimizer.** Model performance across learning rates $\gamma \in \{0.0005, 0.001, 0.005, 0.01\}$ for an unregularized neural network with 50 hidden nodes per layer, trained over 2,000 epochs with a mini-batch size of $M = 16$ (convergence criterion 10^{-7}). (a) Architecture with 1 hidden layer. (b) Architecture with 2 hidden layers. The analytical Runge function is plotted as a reference dashed line. For the single-hidden-layer setup, learning rates $\gamma = 0.005$ and 0.01 yield the best fit. For the two-hidden-layer setup, the best fit is attained using $\gamma = 0.001$.



(a) Single hidden layer.

(b) Two hidden layers.

FIG. 2: **Comparison of learning rates for fitting the Runge function using SGD with RMSprop optimizer.** Model performance across learning rates $\gamma \in \{0.0005, 0.001, 0.005, 0.01\}$ for an unregularized neural network with 50 hidden nodes per layer, trained over 2,000 epochs with a mini-batch size of $M = 16$ (convergence criterion 10^{-7}). (a) Architecture with 1 hidden layer. (b) Architecture with 2 hidden layers. The analytical Runge function is plotted as a reference dashed line. For the single-hidden-layer setup, learning rates $\gamma = 0.005$ and 0.01 yield the best fit. For the two-hidden-layer setup, the best fit is attained using $\gamma = 0.0005$ and 0.001.



(a) Single hidden layer.

(b) Two hidden layers.

FIG. 3: **Testing of activation functions for fitting the Runge function.** Performance of ReLU and Leaky ReLU activations using the Adam optimizer (learning rate $\gamma = 0.001$) for an unregularized neural network with 50 hidden nodes per layer, trained over 2,000 epochs with a mini-batch size of $M = 16$ (convergence criterion 10^{-7}). (a) Architecture with 1 hidden layer. (b) Architecture with 2 hidden layers. The analytical Runge function benchmark is plotted as a reference dashed line. Both ReLU and LeakyReLU fail to produce a good fit.

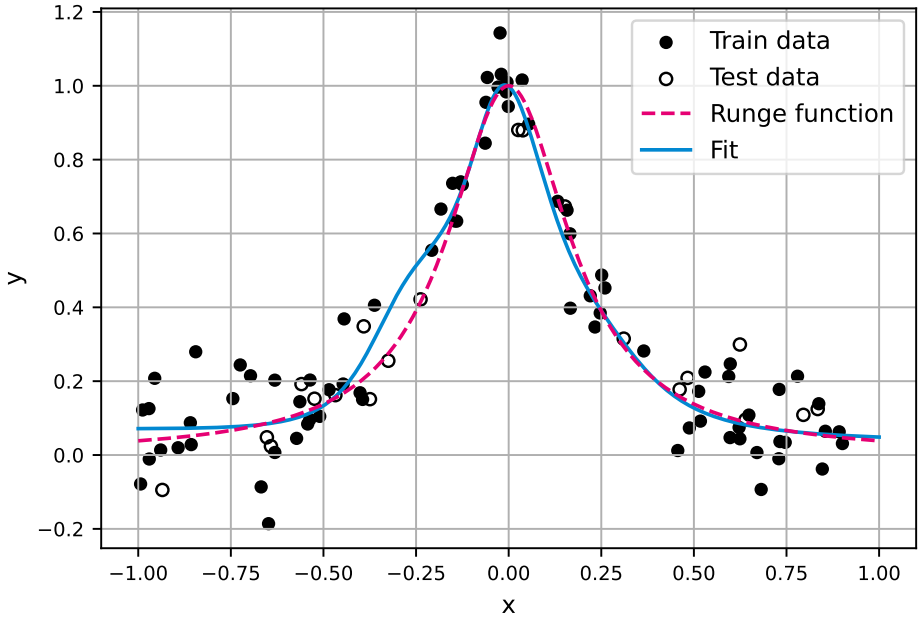


FIG. 4: **Final fit using optimal hyperparameters**. Model performance for fitting the Runge function using the Adam optimizer with no regularization and learning rate $\gamma = 0.001$, on a neural network with 2 hidden layers (50 hidden nodes per layer), trained over 2,000 epochs with a mini-batch size of $M = 16$ (convergence criterion 10^{-7}). The analytical Runge function benchmark is plotted as a reference dashed line.

Cross-Validating the Classifiers

The cross-validation compares a network with one hidden layer of 50 nodes against logistic regression, both trained on 48 000 images in 10 epochs. Tab. IV shows the combinations (λ, γ) within one SE of the best CV accuracy. For both models, three combinations give a CV accuracy within one SE of the best value. For the neural network, the best model is the same as the simplest, $\lambda = 10^{-5}$ and $\gamma = 0.003$, with a CV accuracy of 0.964267 ± 0.000643. For logistic regression, the best and the simplest do not coincide, and we choose the simplest model, $\lambda = 10^{-5}$ and $\gamma = 0.003$, giving a CV accuracy of 0.921800 ± 0.001436. The chosen hyperparameters are then the same for logistic regression and the neural network, which will not always be the case, e.g. if the number of nodes or hidden layers were tuned at the same time. The difference between the two chosen models is $0.964267 - 0.921800 \approx 0.042$, or 4.2 percentage points, and the difference is about the same for the other combinations within one SE. This difference is much larger than the SE, and therefore an actual difference in performance. Logistic regression has about twice the SE of the network, meaning more variation between the folds. But the SE is optimistic. The folds share training data and are therefore not independent, and the same seed is used in every fold, so the SE only reflects the variation from the data split.

We choose the best width for the neural network, given the chosen $\lambda = 10^{-5}$ and $\gamma = 0.003$. A validation curve over the widths $\{5, 10, 25, 50, 100\}$ is shown in Fig. 5. The training accuracy shows how well the model performs on data it has seen, and it grows with the width together with the CV accuracy. At a width of 5, both accuracies are low, and the model is underfitting due to being too simple. At 100 nodes, the gap between the training accuracy and the CV accuracy has grown, meaning the model fits the training data slightly better than unseen data. Since the CV accuracy is also growing, overfitting is not a particular issue. Had the training accuracy improved without the CV accuracy following, the model would be overfitting, and increased complexity would not improve the model. With 10 nodes, the neural network performs within the 1-SE of logistic regression and they can not be separated. With 25 nodes, the network has a higher CV accuracy than logistic regression. This suggests that somewhere between 10 and 25 nodes, the hidden layer starts capturing non-linear relationships that logistic regression can not. 100 nodes give the highest accuracy and are chosen for our final model. The one SE rule does not change the choice here, as the red dotted line in the plot clearly separates 100 nodes from the others. 100 giving a better accuracy than 50 is a further indication that the network is the preferred model over linear regression. Widths above 100 should be tried in further analysis, as the CV accuracy is not yet flat, and the actual best model could have a higher number of nodes.

TABLE IV: **Classification: hyperparameter combinations within one SE of the best CV accuracy for a neural network vs logistic regression.** The network has one hidden layer of 50 nodes. The tested values are $\lambda \in \{10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 0.01\}$ and $\gamma \in \{10^{-4}, 10^{-3}, 3 \cdot 10^{-3}, 0.01, 0.03\}$. Each row is a (λ, γ) combination in the grid with a 5-fold CV accuracy within one SE of the best. The network outperforms logistic regression by around 4 percentage points for all combinations. The one SE rule choose $\lambda = 10^{-5}$ and $\gamma = 0.003$ for both models, which for the network is also the best combination.

λ	γ	CV accuracy	$\pm$ SE
Neural network (50 nodes)			
10^{-6}	0.003	0.964167	0.000639
10^{-5}	0.003	0.964267	0.000643
10^{-6}	0.010	0.964017	0.000900
Logistic regression			
10^{-6}	0.001	0.920833	0.001308
10^{-6}	0.003	0.922117	0.001342
10^{-5}	0.003	0.921800	0.001436

The CV accuracy does not measure the final model, but estimates the expected accuracy of the training procedure, the architecture, λ, γ and 10 epochs of Adam, applied to training sets of 48 000 images. The fold models are discarded, and the final model is a new fit on all 60 000 training images. This model is evaluated in the following.

Evaluating the Classifiers

The model selection and CV accuracies of the previous section indicate that the network is preferred over logistic regression. To verify this for the final models, Tab. V reports the performance of the two, trained on the whole 60 000 images training set and evaluated on the 10 000 test images. The models are compared to a baseline that always guesses 1, the most common digit in the training set [2]. The test accuracy of the baseline is 0.1135, and both models are safe above this, meaning they have learned real structure of the data. The difference in test accuracy between the network and logistic regression is $0.9763 - 0.9250 \approx 0.051$, or 5.1 percentage points. This is larger than the standard error of a test accuracy, $\sqrt{\text{acc}(1 - \text{acc})/N} \approx 0.003$, where acc is the test accuracy and $N = 10\,000$ is the number of test images. Both test accuracies are slightly higher than the corresponding CV accuracies, consistent with the final models being trained on 60 000 instead of 48 000 images. The network predicts better on unseen data, with a test error (1− test accuracy) of 2.4% compared to 7.5% for logistic regression,

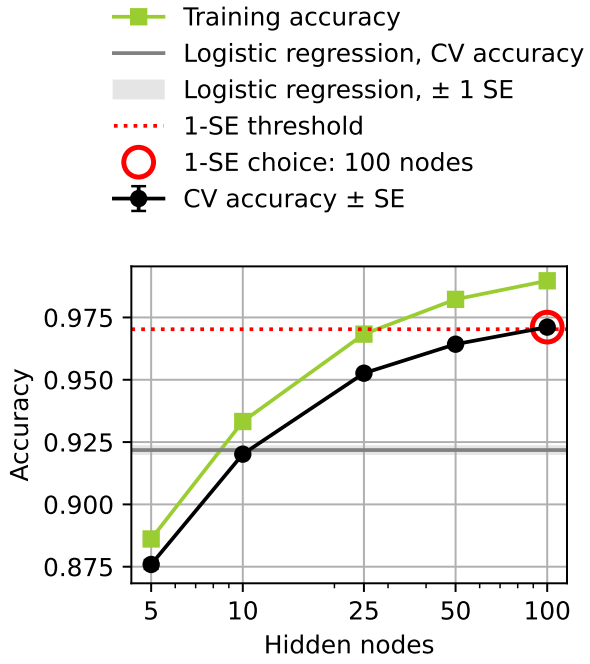


FIG. 5: **Classification: validation curve over the width of a single hidden layer neural network.** $\lambda = 10^{-5}$ and $\gamma = 0.003$ are fixed from the grid search with 50 nodes. The black points show the 5-fold CV accuracy with error bars of one SE, and the green squares the training accuracy, the mean accuracy on the folds the network was trained on. The red dotted line is the 1-SE threshold from the best width (100). The grey line and band show the CV accuracy ± SE of the selected logistic regression model ($\lambda = 10^{-5}$, $\gamma = 0.003$). The CV accuracy increases with width and matches logistic regression at 10 nodes. Only 100 nodes, the widest network tested, lies above its threshold.

but has about ten times as many parameters. The cost of a feed-forward pass is $\mathcal{O}(mK) \approx 7.8 \cdot 10^3$ operations per sample for logistic regression and $\mathcal{O}(mM_1) \approx 7.9 \cdot 10^4$ for the network, about ten times higher. The measured training time is only 1.8 times higher for the network, which might be due to fixed overhead in our implementation dominating the training time of logistic regression. Since it is our only measure of time, we still use it when comparing the models. The final training cross entropy is computed on the training data, and it shows how well the models fit the data they were trained on. It is 0.0387 for the network and 0.2446 for logistic regression, about six times higher, meaning that the network gives a higher probability to the correct class on the training data.

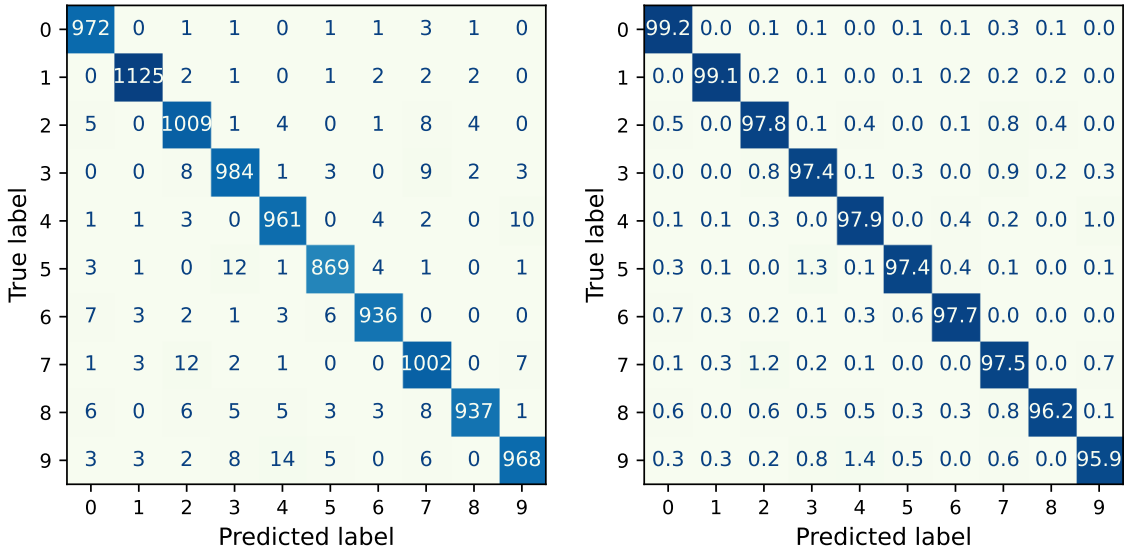
In the following evaluation of the classification, we only focus on the network. Confusion matrices showing counts and row-normalized percentages are presented in Fig. 6. As implied by the test accuracy, the predictions are mostly correct, and the diagonal of correct classifications dominates. The most common confusion is between 9 and 4, which are mistaken for each other in both directions. 7 and 2, and 5 and 3, are also often confused. These digits thus look similar to the model, given the freedom it has had to learn them. The errors are not symmetric. For example, 1.4% of the 9s are predicted as 4, while 1.0% of the 4s are predicted as 9. Similarly, 7 is more often mistaken for 2 than the reverse, and 5 more often for 3.

Other metrics computed from the confusion matrix are presented per class in Tab. VI. The precision is highest for the digits 8, 1 and 6, meaning that very few other digits are predicted as these. 8 has the highest precision, but also quite a low recall. When the model predicts 8, it is very often correct, but true 8s are often predicted as other digits. The lowest precision is for 7, 0.9625, with the errors spread out over 2, 3, 8 and 9 being mistakenly predicted as 7. The highest recall is for 0 and 1, both above 0.99, meaning they are the easiest digits for the model to classify. The lowest recall is for 9, 0.9594, largely because it is confused with 4. The confusion goes both ways, and true 9s are also predicted as several other digits. Measured by the F1-score, however, 9 and 7 are about equally difficult, with 0.9685 and 0.9686, the lowest of all classes.

No classes are critical, as the F1-score lies between 0.9685 and 0.9908, and the lowest recall is 0.9594. The classes of the MNIST set are close to balanced [2], and the test accuracy in Tab. V is thus a good representation of how well the model performs overall. The confusions mostly come in pairs of digits that look similar in shape, meaning that the model has learned the shapes, and that it is mainly the separation of similar shapes that remains difficult. The model could possibly be improved by a wider network, as the CV accuracy in Fig. 5 has not yet flattened at 100 nodes. Replacing the sigmoid with ReLU in the hidden layer could also help, as the sigmoid can give small gradients and slower learning, and there is no reason as to why sigmoid should perform better, as for the regression problem.

When is a Neural Network Worth the Cost

When fitting the Runge function, the neural network achieves performance comparable to polynomial regression using closed-form OLS, though direct score comparison is not possible because a different dataset was used in our previous work. With an appropriate choice of optimizer and hyperparameters, visual inspection confirms a decent fit on the test dataset, although a local deviation is present. With larger datasets, local deviations like this are less likely. Importantly, the neural network avoids the boundary oscillations characteristic of the Runge phenomenon seen in polynomial models. Because the experiment is conducted on a small dataset of 100 points, the computational cost of training the neural network is negligible, though scaling to larger datasets would naturally



(a) Counts.

(b) Row-normalized percentage.

FIG. 6: **Confusion matrix on the test set.** For the network with one hidden layer of 100 nodes ($\lambda = 10^{-5}$, $\gamma = 0.003$). Rows are the true digits and columns the predicted digits, so the diagonal shows the correct classifications. (a) shows counts, and (b) the percentage of each true digit assigned to each predicted digit, so the diagonal of (b) equals the recall in Table VI. The most common errors are 4 and 9 confused in both directions, 5 predicted as 3 and 7 predicted as 2.

TABLE V: **Comparison of the trained classifiers.** Both models are trained on all 60 000 training images and evaluated on the 10 000 test images. Baseline always guesses 1, the majority class in the training set, Logistic is logistic regression, and Network has one hidden layer of 100 nodes, both with $\lambda = 10^{-5}$ and $\gamma = 0.003$. Final cost is the training cross entropy without the penalty after the last epoch, and training time is the wall-clock time for the 10 epochs. The network reaches a test accuracy of 0.9763 compared to 0.9250 for logistic regression, with about ten times as many parameters and 1.8 times the training time.

	Baseline	Logistic	Network
Parameters	–	7 850	79 510
Final cost	–	0.2446	0.0387
Test accuracy	0.1135	0.9250	0.9763
Training time (s)	–	14.3	25.4

TABLE VI: **Metrics per class.** Precision, recall and F1-score for each digit on the test set, for the network with one hidden layer of 100 nodes ($\lambda = 10^{-5}$, $\gamma = 0.003$). Support is the number of test images of each digit. The F1-score ranges from 0.9685 for digit 9 to 0.9908 for digit 1, and the lowest recalls are for 9 and 8.

Class	Precision	Recall	F1-score	Support
0	0.9739	0.9918	0.9828	980
1	0.9903	0.9912	0.9908	1135
2	0.9656	0.9777	0.9716	1032
3	0.9695	0.9743	0.9719	1010
4	0.9707	0.9786	0.9746	982
5	0.9786	0.9742	0.9764	892
6	0.9842	0.9770	0.9806	958
7	0.9625	0.9747	0.9686	1028
8	0.9905	0.9620	0.9760	974
9	0.9778	0.9594	0.9685	1009

increase runtime. While standard polynomial regression suffices for many tasks, for the Runge function investing in a neural network is justified if achieving high accuracy at the boundaries is the primary goal, as that is where polynomial regression typically fails.

For classifying the MNIST handwritten digits, a network with one hidden layer is preferred over logistic regression, because our results say that non-linear relationships are needed to describe the data and reach high accuracies. The comparison is fair in the sense that both models are tuned over the same grid of λ and γ, with the same folds, optimizer and training budget, although the network is also tuned over its width. The computational cost of the network is higher, with a training time of 25.4 s, but not unmanageable. Given the scope of this

project, 10 epochs and minibatches of size 130 are used, reducing the computation time while still yielding satisfactory accuracies. Since the number of epochs and the batch size were not tuned, we do not know how much accuracy is lost by this choice, and it may affect the two models differently. Other settings for gradient descent besides the learning rate, such as the number of epochs, the batch size and the parameters of Adam, could be optimized, and a better balance between cost and results could be determined. The accuracy, together with the confusion matrix and per-class metrics, shows that the model has learned the shapes of the digits, while keeping the training time relatively low. We consider the computational bottleneck of the whole procedure to be the cross-validation, where the grid search alone tried 25 combinations of hyperparameters over 5 folds, giving 125 fits per model, and the validation curve added another 25. We choose the neural network for this particular problem, as it reduces the test error from 7.5% to 2.4%, at less than twice the training time.

V. CONCLUSION

We investigated the properties of feed-forward neural networks used in regression and classification problems. Fitting the one-dimensional Runge function, we explored the effect of varying the number of hidden layers, the activation functions, the form of regularization and the gradient descent methods. We compared against traditional linear regression methods, determining whether a neural network is useful for this problem. The MNIST handwritten digits data set was classified by a neural network and, serving as a baseline, logistic regression. After choosing the best regression and classification models through cross-validation, the models were evaluated and compared to determine if a neural network is worth the cost.

For approximating the one-dimensional Runge function, feedforward neural networks demonstrated performance comparable to standard OLS polynomial regression, successfully avoiding the severe boundary oscillations characteristic of the Runge phenomenon. Optimal performance was achieved using an unregularized two-hidden-layer network trained with the Adam optimizer at a learning rate of $\gamma = 0.005$, using Sigmoid activations, yielding test MSE value 0.0088. Importantly, our results highlight the challenges of working with small datasets. These challenges stem from training set sparsity, which can lead to local deviations during fitting, and the disproportionate influence of individual points in a small test set, where a single high-variance point can skew the fit and make the MSE to appear overly pessimistic or optimistic.

Cross-validation over a grid gave $\lambda = 10^{-5}$ and $\gamma = 0.003$ for both classifiers by the one SE rule, with a CV accuracy 4.2 percentage points higher for the network than for logistic regression, much larger than the SE. A validation curve over the width of the network showed that its CV accuracy surpassed that of logistic regression between 10 and 25 nodes, and that 100 nodes gave the highest accuracy. Trained on the whole training set and evaluated on the test set, the network reached an accuracy of 0.9763, while logistic regression reached 0.9250. The network reduces the test error from 7.5% to 2.4%, removing about two thirds of the errors made by logistic regression. No digit was critical for the network, with F1-scores between 0.9685 and 0.9908. The errors were mostly confusions between digits of similar shape, such as 4 and 9, 7 and 2, and 5 and 3, suggesting that the network has learned the shapes of the digits, and that mainly the separation of similar shapes remains difficult.

The higher accuracy comes at a cost. The network has about ten times as many parameters as logistic regression, and its estimated cost of a feed-forward pass is about ten times higher. The measured training time was only 1.8 times higher, 25.4 s compared to 14.3 s, which might be due to fixed overhead in our implementation. The network also fits the training data better, with a final training cross entropy about six times lower than that of logistic regression. Our conclusion is that a network with one hidden layer is preferred over logistic regression for classifying the MNIST digits, suggesting that nonlinear relationships are needed to describe these data. For the regression task, training cost remained negligible due to the small size of the Runge dataset, though computational cost would naturally increase with larger datasets. While standard polynomial regression is cheap and effective for many tasks, avoiding the severe boundary oscillations seen in the Runge function justifies the added cost of a neural network.

Though our results have provided insight into the strengths and weaknesses of neural networks, our analysis lacks some useful investigations. For the regression analysis, future work should extend to larger datasets to address the high statistical variance and data sparsity observed with small datasets. When cross-validating the classifiers, the same seed was used in every fold, so the SE only reflects the variation from the data split. Repeating the cross-validation over multiple seeds would give a more realistic SE and show whether the choice of hyperparameters is robust. Widths above 100 should be tried in further analysis of the classification, as the CV accuracy had not yet flattened and 100 nodes gave the highest accuracy. The number of epochs and the batch size were not tuned, and a better measure of the training time should be used. Also, the sigmoid activation function was used in the hidden layer throughout for classification, and should be compared against ReLU for insight into which function works better for this problem.

REFERENCES

[1] Ribana Roscher, Bastian Bohn, Marco F. Duarte, and Jochen Garcke. Explainable machine learning for scientific insights and discoveries. *IEEE Access*, 2020.

[2] Yann LeCun, Corinna Cortes, and CJ Burges. Mnist handwritten digit database. *ATT Labs [Online]. Available: http://yann.lecun.com/exdb/mnist*, 2, 2010.

[3] Morten Hjorth-Jensen. Applied data analysis and machine learning: Lecture notes. https://educationalmaterialuio.github.io/MachineLearningUiO/doc/LectureNotes/_build/html/intro.html, 2025. Department of Physics, University of Oslo. Accessed October and November 2026.

[4] Sebastian Raschka, Yuxi (Hayden) Liu, and Vahid Mirjalili. *Machine Learning with PyTorch and Scikit-Learn*. Packt Publishing, Birmingham, UK, 2022.

[5] Tomasz Szandala. Review and comparison of commonly used activation functions for deep neural networks. *arXiv:2010.09458*, 2020.

[6] Klara Elvira Boström, Vanii Hsin-Ni Yao, Catherine Hughes, and Shahnaz Jurat. Closed-form and gradient-based methods for polynomial regression: Benchmarking ols, ridge, and lasso on the runge function, 2026. Project 1 report, FYS-STK4155, University of Oslo.

[7] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.

[8] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[9] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.

[10] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[11] Michael L. Waskom. seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60):3021, 2021.

[12] Wes McKinney. Data Structures for Statistical Computing in Python. In Stéfan van der Walt and Jarrod Millman, editors, *Proceedings of the 9th Python in Science Conference*, pages 56 – 61, 2010.

[13] EducationalMaterialUiO. MachineLearningUiO: Educational material for machine learning. https://github.com/EducationalMaterialUiO/MachineLearningUiO, 2025. GitHub repository. Accessed October and November 2026.

[14] OpenAI. ChatGPT. https://chatgpt.com/, 2026. GPT-6 Luna, accessed October and November 2026.

[15] Anthropic. Claude (Claude Sonnet 5). https://claude.ai, 2026. Accessed October and November 2026.

[16] Google. Google Gemini. https://gemini.google.com/, 2026. 3.8 Flash, October and November 2026.

APPENDICES

Appendix A: Derivation of the backpropagation equations by the chain rule

The gradient in Eq. 16 is a sum over samples plus a penalty term. Here, we derive the gradient for a single sample, without the penalty. In this appendix, C denotes the per-sample cost $\mathcal{L}(\mathbf{y}_s, \hat{\mathbf{y}}_s)$, and we omit the index s.

In the forward pass we have Eq. 1

$$z_j^{(l)} = \sum_i w_{ij}^{(l)} a_i^{(l-1)} + b_j^{(l)}, \qquad a_j^{(l)} = f\left(z_j^{(l)}\right).$$

The partial derivatives needed are

$$\frac{\partial z_j^{(l)}}{\partial w_{ij}^{(l)}} = a_i^{(l-1)}, \qquad \frac{\partial z_j^{(l)}}{\partial b_j^{(l)}} = 1, \qquad \frac{\partial z_k^{(l+1)}}{\partial a_j^{(l)}} = w_{jk}^{(l+1)}. \tag{A1}$$

The parameters $w_{ij}^{(l)}$ and $b_j^{(l)}$ affect the cost by being part of $z_j^{(l)}$. The chain rule then gives

$$\frac{\partial C}{\partial w_{ij}^{(l)}} = \frac{\partial C}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} a_i^{(l-1)},$$
$$\frac{\partial C}{\partial b_j^{(l)}} = \frac{\partial C}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial b_j^{(l)}} = \delta_j^{(l)},$$

which are Eqs. 20 and 19. We then need to find what $\delta_j^{(l)}$ is in each layer. The net input $z_j^{(L)}$ affects the cost through $a_j^{(L)}$, so

$$\delta_j^{(L)} = \frac{\partial C}{\partial a_j^{(L)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} = \frac{\partial C}{\partial a_j^{(L)}} f'\left(z_j^{(L)}\right).$$

Collecting all neurons in a vector gives Eq. 17. In a hidden layer, $z_j^{(l)}$ affects $a_j^{(l)}$, and $a_j^{(l)}$ in turn affects every net input $z_k^{(l+1)}$ in the next layer. The chain rule therefore contains a sum over k,

$$\begin{aligned}\delta_j^{(l)} &= \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \sum_k \frac{\partial C}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial a_j^{(l)}} \\ &= f'\left(z_j^{(l)}\right) \sum_k w_{jk}^{(l+1)} \delta_k^{(l+1)},\end{aligned}$$

where we have inserted Eq. A1. The sum is element j of the matrix-vector product $\mathbf{W}^{(l+1)}\boldsymbol{\delta}^{(l+1)}$, which gives Eq. 18.

Appendix B: Derivation of the cost function gradients

We derive the derivatives of the per-sample cost functions $\mathcal{L}$ in Eq. 10 with respect to the network output. For regression, the output of sample s is the scalar $\hat{y}_s$, and the MSE gives

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}_s} = \frac{\partial}{\partial \hat{y}_s}(\hat{y}_s - y_s)^2 = 2(\hat{y}_s - y_s).$$

For classification, only the term with $k' = k$ in the sum depends on p_{sk}, so the cross-entropy gives

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial p_{sk}} = -\frac{\partial}{\partial p_{sk}} \sum_{k'=0}^{K-1} y_{sk'} \log p_{sk'} = -\frac{y_{sk}}{p_{sk}}.$$

Since Ω does not depend on the network output, inserting these into Eq. 10 gives

$$\frac{\partial C_{\text{MSE}}}{\partial \hat{y}_s} = \frac{2}{n}(\hat{y}_s - y_s), \qquad \frac{\partial C_{\text{CE}}}{\partial p_{sk}} = -\frac{1}{n}\frac{y_{sk}}{p_{sk}}.$$

The regularization term only contributes to the derivatives with respect to the weights,

$$\frac{\partial C}{\partial w_{ij}^{(l)}} = \frac{1}{n}\sum_{s=1}^{n} \frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}} + \lambda \begin{cases} 2w_{ij}^{(l)}, & \ell_2, \\ \operatorname{sgn}\left(w_{ij}^{(l)}\right), & \ell_1, \end{cases}$$

while

$$\frac{\partial C}{\partial b_{j}^{(l)}} = \frac{1}{n}\sum_{s=1}^{n} \frac{\partial \mathcal{L}}{\partial b_{j}^{(l)}}.$$

Setting $\lambda = 0$ gives the unregularized case. The derivatives of $\mathcal{L}$ with respect to the weights and biases are computed by backpropagation.

Appendix C: Use of AI/LLM Tools

Our use of Artificial Intelligence (AI) and Large Language Model (LLM) tools are documented in this appendix. In the list below, together with Tabs. VIII and VII, we declare our use of AI. In Tabs. IX and X the system for this declaration is presented, these tables are taken from the course FYS-STK4155 [13] at UiO. For the declarations of using level 1 in writing text, an example is provided on the last page of the report.

Tools used

- ChatGPT[14] (GPT-6 Luna, accessed via chatgpt.com, October and November 2026).
- Claude[15] (Claude Opus 5.5, accessed via claude.ai, October and November 2026).
- Google Gemini [16] (3.8 Flash, accessed via gemini.google.com, October and November 2026).

AI/LLM Contribution

TABLE VII: Use of AI/LLM in coding.

File	LLM contribution level	Description
`cv_digits.py`	1	Debugging.
`estimator.py`	1	Debugging.
`evaluate_digits.py`	2	- Claude wrote the function `plot_grid`, given our data frame structure, our function returning the 1 SE choice and an example heatmap made in weekly exercises. - Claude assisted in visualizing the validation curve in the function `plot_width` by adding the ± SE band and marking the chosen width. - Claude assisted in plotting the confusion matrices, by altering independently written plot to include percentages and include more digits without loosing readeability of axes.
`neural_network.py`	1	Debugging.
`regression.py`	1	Debugging, formatting of plots.
`test_NN.py`	0	Written independently.
`test_classification.py`	1	Debugging.
`utils.py`	2	Claude altered independently written `train` function from only including plain GD to also include our previous implementation of RMSProp and Adam from project 1 [6]. The setup using the neural network was done independently before using Claude.
`.gitignore`	0	Made by https://www.toptal.com/developers/gitignore/
`README.md`	0	Written independently.

TABLE VIII: Use of LLM in writing and editing.

Section	Level	Description
Abstract	0	Written independently.
Introduction	1	Minor language corrections.
Theory		
Feed-Forward Neural Networks	1	Rephrasing for clarity.
Activation Functions	1	Rephrasing for clarity.
Cost Functions	1	Checked notation, improved clarity.
Backpropagation	1	Checked notation, improved clarity.
Logistic Regression	0	Written independently.
Classification Metrics	0	Written independently.
Cross-validation and Model Selection	1	Minor language corrections.
Methods		
Initialization of the Neural Network	0	Written independently.
Data sets	0	Written independently.
Model Selection for the Regression Problem	1	Rephrasing for clarity.
Cross-validation and Model Selection	0	Written independently.
After Choosing the Models	0	Written independently.
Tools	0	Written independently.
Results and Discussion		
Optimizer choice for regression	1	Improved clarity and flow.
Choice of activation function for regression	1	Rephrasing for clarity.
Choice of hyperparameters for regression	1	Rephrasing for clarity.
Final fit on the Runge data	1	Rephrasing for clarity.
Cross-validating the Classifiers	1	Rephrasing for clarity.
Evaluating the Classifiers	1	Rephrasing for clarity.
When is a Neural Network Worth the Cost	1	Rephrasing for clarity.
Conclusion	1	Minor language corrections.
Appendices		
A: Backpropagation derivation	0	Written independently.
B: Cost function gradients	0	Written independently.
C: Use of AI/LLM Tools	0	Written independently.

TABLE IX: Classification of LLM assistance in code development.

Code level	Label	Meaning
0	None	No LLM assistance.
1	Debugging	LLM identified a bug; you implemented the fix.
2	Snippet	LLM provided a function, loop or short block; you integrated and tested it.
3	Skeleton	LLM generated the structure of a script/class; you filled in the logic.
4	Substantial	LLM wrote most of a file; you adapted, tested and commented it.

TABLE X: Classification of LLM assistance in written text.

Text level	Label	Meaning
0	None	No LLM assistance.
1	Language only	Spell-check, grammar, minor phrasing.
2	Editorial	LLM rewrote or restructured your sentences; you supplied the ideas and checked the result.
3	Skeleton	LLM drafted substantial text from your prompts; you revised, verified and own it.

Example of use for improving the flow of a section, level 1:

Old version: Confusion matrices showing counts and row-normalized percentages are presented in Fig. 6. The predictions are, as implied by the test accuracy, mostly correct and the diagonal with correct classifications dominate. The most mixed up classified digits are 9 and 4, being mistaken for each other in both ways. Also often confused are 7 and 2, and 5 and 3. These are thus digits that look similar to the model, given the freedom it has had to learn them. The errors are not the same in both ways, e.g. is 9 mistaken for being 4 in 1.4% of the cases, while 4 is mistaken for 9 in 1.0% of the cases. 7 is confused for being 2 and 5 is confused for being 3.

New version, generated by Claude Opus 5.5 (November 2026): Confusion matrices showing counts and row-normalized percentages are presented in Fig. 6. As implied by the test accuracy, the predictions are mostly correct, and the diagonal of correct classifications dominates. The most common confusion is between 9 and 4, which are mistaken for each other in both directions. 7 and 2, and 5 and 3, are also often confused. These digits thus look similar to the model, given the freedom it has had to learn them. The errors are not symmetric. For example, 1.4% of the 9s are predicted as 4, while 1.0% of the 4s are predicted as 9. Similarly, 7 is more often mistaken for 2 than the reverse, and 5 more often for 3.

What the LLM changed: Claude was given the original text together with the prompt "im analyzing a classifier classifying digits. can you improve the clarity and flow of this text, do not alter the contents. you are allowed to change the wording and grammar". It then changed some sentence structures and wordings, making it clearer what the author wants to convey. The author fact checked the output, made sure that the new text indeed tells the same as the original text and pasted it in the report, replacing the old version. The new text is grammatically correct and separates the information into smaller and more easily digesteable pieces, with clearer wording. The generated text might be altered again by the author.